Fundamentos de Programación

Tema 4: Programación Modular. Funciones en Python



Docente 2025/2026: Daniel Castro Pereiro
Apuntes originales: Efrén Arias Jordán y David Mera Pérez

Dpto Electrónica e Computación

Concepto de Subprograma

- Podemos pensar que un programa se compone de varias partes, por ejemplo:
  - **obtener los datos de entrada** por parte del usuario
  - **realizar** ciertos **cálculos** con los mismos
  - **mostrar el resultado** de esos cálculos en la pantalla.
- Se trata de descomponer la tarea a realizar en unas cuantas subtareas, que a su vez pueden descomponerse en subtareas más pequeñas o más simples y así sucesivamente. Llegará un momento en que dichas subtareas serán lo suficientemente pequeñas como para que sea sencillo programarlas.
- Este método se conoce como **diseño descendente** y da lugar a la **programación modular**.
- Un **subprograma** o una **función** es un fragmento de un código que resuelve una tarea con entidad propia
- El uso de subprogramas facilita las tareas de la programación
  - Permite dividir un programa en componentes más pequeños, lo que implica **reutilización**
  - Los programas son más **claros**, **modificables** y **reutilizables**
  - Son más fáciles de **diseñar** y **verificar**
  - Se puede **distribuir el trabajo** de programación entre varias personas
  - El uso de subprogramas permite la **descomposición** y desarrollo **modular**.
- Ya hemos usado **módulos**, es decir, ficheros que contienen funciones y variables de valor predefinido que podemos importar en nuestros programas.

Funciones

- Podemos usar:
  - **Subprogramas predefinidos**

```
input(), print(), type(), abs()…
```

  - **Subprogramas en librerías**
  - **Subprogramas creados** por el programador
- En el caso de Python estos subprogramas se conocen como **funciones**.
- Denominaremos **activar**, **invocar** o **llamar** a una función a la acción de usarla. Las funciones que hemos aprendido a invocar reciben cero, uno o más argumentos separados por comas y encerrados entre un par de paréntesis y que, habitualmente, devuelven un valor.

```
>>> abs(-3)
3
```

```
>>> abs(round(2.45,1))
2.5
```

- Podemos llamar a una función desde una expresión. Como el resultado tiene un tipo determinado, hemos de estar atentos a que éste sea compatible con la operación y tipo de los operandos con los que se combina:

```
>>> 1+(abs(-3)*2)
7
>>> 2.5 / abs(round(2.45,1))
1.0
>>> 3+str(3)
Traceback (most recent call last):
File "<stdin>", line 1, in ?
TypeError: number coercion failed
```

- Los argumentos de una función también pueden ser expresiones:

```
>>> abs(round(1.0/9, int(4/(1+1))))
0.11
```

- Eso sí, los argumentos deben coincidir con el tipo del parámetro que se espera (es por esto que se convierte a **`int`** el segundo parámetro de la llamada a **`round`**).

Funciones

- En el siguiente ejemplo vemos cómo la función **`round`** funciona correctamente cuando se le pasa un argumento del tipo adecuado (**`float`**) y cómo se produce un error cuando se le pasa un **`str`** en lugar de un **`float`**.

```
>>> round(7.3)
7.0
>>> round('7.3')
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
TypeError: a float is required
```

Módulos y librerías

- Python proporciona además funciones trigonométricas, logaritmos, etc., pero no están directamente disponibles cuando iniciamos una sesión (aunque algunos entornos de desarrollo incluyen por defecto las más usadas).

- Un **módulo** (o **librería**) es un conjunto de definiciones de variables, funciones y tipos de datos (entre otras cosas) que pueden importarse para usarlas en nuestros programas.

- Antes de utilizarlas debemos **importar el módulo** que las contiene, consiguiendo así que Python las reconozca. Empezaremos por importar el módulo matemático (**`math`**):

```
>>> import math
```

- Ahora podemos utilizar la función seno (**`sin`**, del inglés sine, indicándole los radianes) en nuestros cálculos.

```
>>> math.sin(0)
0.0
>>> math.sin(1)
0.841470984808
```

Funciones

- Son muchos los módulos existentes para trabajar en Python. Por ejemplo, el módulo **`math`** contiene las funciones matemáticas clásicas y **`random`** funciones que generan números pseudoaleatorios.
- Hay otros módulos que son más específicos y se centran en problemas concretos como, por ejemplo:

Interfaces gráficas wxPython, pyGtk, pyQT,tkinter, …
Bases de datos MySQLdb, pySQLite, cx_Oracle,sqlite3…
Imagen PIL, gdmodule, VideoCapture,…
Ciencias scipy, numarray, NumPy,…
Videojuegos Pygame, Soya 3D, pyOpenGL,…
Sonido pySonic, pyMedia, pyMIDI,…

- Existen también módulos de **geolocalización, de puertos** (USB, serie, paralelo, . . .), para programación de **dispositivos móviles**, **Web**, **programas de mensajería** y casi para cualquier cosa que podamos imaginarnos.

Tipos de módulos:

- **Módulos de la librería estándar** de Python (incorporados en el intérprete). Puedes consultar el listado con la ayuda en: https://docs.python.org/3/library/
- **Módulos disponibles en internet** que se pueden instalar
- **Nuestros propios módulos.**

Funciones

Acceso a la funcionalidad de un módulo

- Tenemos tres opciones:
  - **Importar el módulo** en cuestión y acceder a la función utilizando el nombre del módulo (recomendable cuando usamos funciones con el mismo nombre en distintos módulos):

```
>>> import math # importa modulo math
>>> print (math.sin(2.0)) # visualiza el seno(2.0)
```

  - **Importar la función** del módulo y utilizarla directamente:

```
>>> from math import sin
>>> print (sin(2.0))
```

  - **Importar todas las funciones** del módulo y utilizarlas directamente (usaremos esta opción):

```
>>> from math import *
>>> print (sin(2.0))
```

- Si un programa utiliza muchos módulos, puede ocurrir que el mismo nombre de función este en varios módulos y el programa no sabe a qué función se refiere (Se usa la función del último módulo que se invoca en el programa). Para deshacer esta ambigüedad será preferible la primera opción.

Funciones de la biblioteca `math`

- **`math`** : con diversas funciones matemáticas para números **`float`**. Para números complejos y cambio de sistemas de coordenadas (cartesianas y polares) **`cmath`**

```
sin(x),cos(x),tan(x),asin(x),atan(x),acos(x) #Seno, coseno
, tangente, arcoseno, arcocoseno, arcotangente de x

sinh(x),cosh(x),tanh(x),asinh(x),atanh(x),acosh(x) #Igual
que el anterior con seno hiperbólico de x

degrees(x),radians(x) #Convierte ángulo x de grados a radi
anes y viceversa.

Constantes pi y e #Constante matemática pi=3.14 e o número
 e=2.7

exp(x),log(x),log10(x) #Función exponencial (e**x), logari
tmo neperiano y logaritmo en base 10 de x.

sqrt(x) #Raíz cuadrada de x.
```

Funciones

```
floor(x) #Entero mas grande que sea menor o igual que el float x

round(x) #Entero más próximo al float x

ceil(x) #Entero más pequeño mayor o igual que el float x

trunc(x) #Devuelve la parte entera del float x

fabs(x) #Valor absoluto del float x

factorial(x) #Factorial del entero positivo x

isnan(x) #Comprueba si x es NaN (Not a Number)

isinf(x) #Comprueba si x es infinito

max(x1,x2,x3,...) #Devuelve máximo de los argumentos

min(x1,x2,x3,...) #Devuelve mínimo de los argumentos
```

Funciones de la biblioteca `random`

- Genera números pseudo-aleatorios para distintas distribuciones.

```
seed(a=None) #inicializa el estado interno del generador de números aleatorios. Con None o sin argumento, el generador se inicializa normalmente con el reloj del sistema operativo.

random() #devuelve un número aleatorio en el intervalo [0.0, 1.0).

uniform(a,b) #devuelve un número aleatorio  float N tal que a <= N <= b

randint(a,b) : #devuelve un número aleatorio entero N tal que a <= N <= b.

randrange(inicio,final[,paso]): #devuelve un elemento aleatoriamente seleccionado de la secuencia generada con range(inicio,final,paso).

choice(seq) #devuelve un elemento aleatorio de una secuencia que no está vacía. Si la secuencia está vacía eleva una excepción IndexError.
```

Funciones

```
shuffle(x[,random]) #baraja la secuencia x.

sample(population,k) #devuelve una lista de longitud k
con elementos aleatorios de la secuencia population.

gauss(media,desviacion) #Distribución gaussiana.

normalvariate(media,desviacion) #Distribución normal.
```

Funciones de la biblioteca `time`

- Este módulo permite realizar algunas operaciones básicas de fecha y hora.

```
time.time() #Devuelve el número de segundos transcurrid
os desde 01/01/1970.

time.ctime() #Convierte un número que corresponde a los
 segundos transcurridos desde 01/01/1970 en la fecha co
rrespondiente.

time.gmtime() #Devuelve una estructura de tupla de tiem
po con la hora del Meridiano de  Greenwich (GMT).

time.localtime() #Devuelve una estructura de
tupla de tiempo con la hora del huso horario
local del sistema.

time.asctime() #Devuelve una cadena de caracteres con
 la fecha que se introduce como una tupla de tiempo.

time.sleep() #Permite detener la ejecución  de un scr
ipt durante un tiempo definido en  segundos.
```

El módulo `datetime`.

- Este módulo contiene clases relativas al manejo de fechas. A diferencia del módulo **`time`**, los datos de tiempo representan atributos.

```
datetime.time #Corresponde a una clase que
contiene datos de horas como atributos.

datetime.date #Corresponde a una clase que
contiene datos de fechas como atributos.

datetime.datetime #Corresponde a una clase que contie
ne datos de fechas y horas como atributos.
```

Funciones

```
datetime.timedelta #Corresponde a una clase que puede m
anipular diferencias horarias.

datetime.now() #fecha y hora actual
```

Funciones de la biblioteca sys

- Proporciona acceso a funcionalidades usadas por el intérprete

```
sys.argv #la lista de argumentos de la línea de comando
s de un programa Python. argv[0] es el nombre del progr
ama, argv[1] el primer argumento, ...

sys.maxint #devuelve el entero más grande que usa Pytho
n. EN PYTHON 2

sys.maxsize #devuelve la longitud máxima de los contene
dores de Python.

sys.platform #contiene el tipo de plataforma usada.

exit() #finaliza un programa.
```

Funciones de la biblioteca os

- Funcionalidad dependiente del sistema operativo. Funciones para trabajar de manera portable con archivos y directorios

```
access(path, modo_de_acceso) #saber si se puede acceder
 al archivo o directorio path.

getcwd() #devuelve el direct. actual.

chdir(nuevo_path) #cambia de directorio de  trabajo a n
uevo_path.

mkdir(path[, modo]) #crea un direct. de nombre path.

rmdir(path) #elimina el direct. path.

rename(actual, nuevo) #renombra un archivo.

chroot() #cambia al directorio raíz

remove(path) #Eliminar un archivo

listdir(path) #contenido del directorio path
```

Funciones

- Contiene el diccionario **environ** con las variables de entorno relativas al sistema:

```
import os
for variable, valor in os.environ.iteritems():
    print('%s: %s' %(variable,valor))
```

- El submódulo **os.path** permite acceder a funcionalidades relacionadas con el nombre de los archivos y directorios:

```
exists(path) #devuelve True se existe path y False en caso
 contrario.

isfile(path) #devuelve True si path es un archivo y False
en caso contrario.

isdir(path) #devuelve True si path es un directorio y Fals
e en caso contrario.
```

- Otras bibliotecas estándar
  - **pdb** : para depurar un programa
  - **string** : manejo de cadenas de caracteres
  - **pandas**: análisis de datos

Definición de nuevas funciones

- Vamos a estudiar el modo en que podemos definir nuestras propias funciones en Python para invocarlas posteriormente.

Definición y uso de funciones con un solo parámetro

- Empezaremos definiendo una función muy sencilla, que recibe un número y devuelve el cuadrado de dicho número. El nombre que daremos a la función es **cuadrado**.

```
def cuadrado(x):
    return x**2
```

- Una vez que hayamos definido la función podremos usarla (invocarla o llamarla) de la misma forma que las funciones ya definidas en Python

Funciones

- Si intentamos ejecutar este programa:

```
def cuadrado(x):
    return x**2
```

- no ocurrirá nada en absoluto; bueno, al menos nada que aparezca por pantalla. Este otro programa sí muestra algo por pantalla:

```
def cuadrado(x):
    return x**2

a = cuadrado(2)
print(a)
4
```

```
#Tema4_01
def cuadrado(x):
    return x**2

a = cuadrado(2)
print(a)
b = 1 + cuadrado(3)
print(cuadrado(b * 3))
```

- Al invocar la función **`cuadrado`** se ejecuta ésta. En el programa, la invocación de la última línea provoca la ejecución de la línea 2 con un valor de **x** (**parámetro formal**) igual a 2 (**parámetro real**).
- El valor devuelto con **`return`** es almacenado en la variable **a** como efecto de la asignación y, posteriormente mostrado en pantalla a causa del **`print`**
- La definición de una función sólo hace que Python aprenda silenciosamente un método de cálculo asociado al identificador **`cuadrado`**. Nada más.
- *Definir una función sólo asocia un método de cálculo a un identificador y no supone ejecutar dicho método de cálculo.*

Funciones

- Vamos con un ejemplo más: una función que calcula el valor de x por el seno de x:

```
from math import sin
def xsin (x):
    return x * sin(x)
```

- Lo interesante de este ejemplo es que la función definida, **xsin**, contiene una llamada a otra función (**sin**). No hay problema: desde una función puedes invocar a cualquier otra.

- Supongamos que definimos una función con un parámetro x como esta:

```
def cubo (x):
    return x**3
```

- Uno de los problemas habituales, es confundir el parámetro **x** con una variable **x**. Por tanto, parece extraño que podamos invocar así a la función:

```
def cubo (x):
    return x**3

y=1
el_cubo = cubo(y)
print(el_cubo)
```

- ¿Cómo es que ahora llamamos **y** a lo que se llamaba **x**? No hay problema alguno. Al definir una función, usamos un identificador cualquiera para referirnos al parámetro (**parámetro formal**), y al invocarla lo hacemos con nuestra variable (**parámetro real o argumento**). Tanto da que se llame **x** como **y.**

- En el cuerpo de una función no sólo pueden aparecer sentencias **return**, también podemos usar estructuras de control: condicionales, bucles, etc.

Funciones

- Lo podemos comprobar diseñando una función **es_mayor_de_edad** que recibe un número (edad) y devuelve un booleano.

```
#Tema4_02. Solucion 1
def es_mayor_de_edad(edad):
    if edad < 18:
        resultado = False
    else:
        resultado = True
    return resultado

#Solucion 2
def es_mayor_de_edad(edad):
    if edad < 18:
        return False
    else:
        return True

#Solucion 3
def es_mayor_de_edad(edad):
     return edad>=18
```

- ***Ejemplo**: Diseñar una función que nos diga si un número dado es **perfecto**.*
- Se dice que un número es perfecto si es igual a la suma de todos sus divisores excluido él mismo. Por ejemplo, 28 es un número perfecto

```
#Tema4_03.
def es_perfecto(n):
    if n==1:
        return True
    sumatorio=0
    for i in range (1,n//2+1):
        if n%i==0:
            sumatorio+=i
    return sumatorio==n

x=int(input('dame el valor del número '))

if es_perfecto(x):
      print('es perfecto')
else:
      print('NO es perfecto')
```

Funciones

- Los programas que diseñemos a partir de ahora tendrán tres «tipos de línea»:
  - ***importación de módulos (o funciones y variables de módulos)***
  - ***definición de funciones***
  - ***sentencias del programa principal.***
- En principio puedes alternar líneas de los tres tipos. Pero compara estos ejemplos (es mucho más legible el primero):

```
#Tema4_04. Primera forma
from math import sin,pi

def cubo (x):
    return x**3

def xsin (x):
    return x * sin(cubo(x))

x=float(input('dame el valor del ángulo '))

print(xsin(x*pi/180))
```

```
#Tema4_05. Con este otro. Segunda forma

x=float(input('dame el valor del ángulo '))

def xsin (x):
    return x * sin(cubo(x))

def cubo (x):
    return x**3

from math import sin,pi
print(xsin(x*pi/180))
```

Definición y uso de funciones con varios parámetros

- No todas las funciones tienen un sólo parámetro.

```
def area_rectangulo(altura, anchura):
    return altura*anchura
#parámetros formales separados por ,
area = area_rectangulo(3,4)
#área es igual a la multiplicación de los parámetros reales
print(area)
```

Definición y uso de funciones SIN parámetros

- **Los paréntesis son obligatorios al crearla y al invocarla**

```
def lee_entero():
    return int(input())

a = lee_entero()
print(a)


#Tema4_06
def lee_entero_positivo():
    numero = int(input("Dame un número positivo "))
   while numero < 0:
       print("El número debe ser positivo")
       numero = int(input("Dame un número positivo "))
   return numero

a = lee_entero_positivo()
 print(a)
```

- Una posible aplicación de las funciones sin argumentos es la presentación de menús con selección de opción por teclado.

- Esta función, muestra un menú con tres opciones, pide al usuario que seleccione una y se asegura de que la opción seleccionada es válida. Si el usuario se equivoca, se le informa del error:

```
#Tema4_07 (mejorar con el uso del try except Ver
Tema4_07_1)

def menu():
    opcion = ''
    while not ('a' <= opcion <= 'c'):
        print("\tCajero automático:")
        print("\t\ta) Ingresar dinero")
        print("\t\tb) Sacar dinero")
        print("\t\tc) Consultar saldo")
        opcion = input("\t\tElige una alternativa:")
        if not 'a' <= opcion <= 'c':
            print("Elige una alternativa correcta: a, b o
 c:")
    return opcion

accion = menu()
print(accion)
```

Procedimientos: funciones sin devolución de valor

- Una función que no devuelve un valor se denomina **procedimiento**. Es una función sin valor de retorno.
- ¿Y para qué sirve una función que no devuelve nada? Bueno, puede, por ejemplo, mostrar mensajes o resultados por pantalla.
- No te equivoques: **mostrar algo por pantalla no es devolver nada.** Mostrar un mensaje por pantalla es un efecto colateral.
- Un procedimiento es un mecanismo para describir una tarea (o una acción) que no produce un resultado y que se puede invocar o llamar para realizar dicha tarea (o acción).
- Al contrario de lo que ocurre con las funciones que retornan un valor, la invocación a un procedimiento (función sin retorno) **nunca se puede realizar dentro de una expresión**

```
#Tema4_08. procedimiento
def tabla_del(n):
    for i in range(1,11):
        print(n,"x",i,"=",n*i)

# programa principal
tabla_del(7)
```

- Otro ejemplo

```
    #Tema4_09
def es_perfecto(n):
    if n==1:
       return True
    sumatorio=0
    for i in range (1,n//2+1):
        if n%i==0:
            sumatorio+=i
    return sumatorio==n

def tabla_perfectos(m):
    for i in range(1,m+1):
        if es_perfecto(i):
            print(i, 'es perfecto')

x=int(input('dame el valor del número '))
tabla_perfectos(x)
```

Funciones

- Supón que te piden que diseñes una función que reciba un entero y devuelva su última cifra. Te piden esto:

```
def ultima_cifra(n):
    return n%10
#No te piden esto otro:
def ultima_cifra(n):
    print(n%10)
```

- Fíjate en que la segunda definición hace que la función no pueda usarse en expresiones como esta:

```
a=ultima_cifra(10293)+1
```

Los procedimientos aceptan el uso de la sentencia **`return`** aunque, eso sí, sin expresión alguna a continuación: Aborta inmediatamente la ejecución de la llamada a la función. Es, como una sentencia **`break`** en un bucle, pero asociada a la ejecución de una función.

Valores de retorno

- El concepto de procedimiento es común, desde un punto de vista conceptual, a los diferentes lenguajes PERO Es importante hacer notar que **las funciones siempre devuelven un valor**, incluso cuando aparentemente no devuelven nada. Este valor es conocido como **`None`**.
- Así, cuando una función no tiene una instrucción **`return`**, o cuando hay una instrucción **`return`** que no tiene ninguna expresión a continuación, se está devolviendo **`None`**.

```
def show(n):
    print(n)
print(show(5))
 5
 None
```

Funciones

```
#Tema4_10 Ver también Tema4_10_1
def divide(x, y):
    try:
        result = x / y
    except ZeroDivisionError:
        print("division por cero!")
    except TypeError:
        print('Valor de división invalido' )
    else:
        print("el resultado es", result)
    finally:
        print("Ejecutando la clausula finally")

divide(2, 1)
input() #para que se pare la ejecucion
divide(2, 0)
input() #para que se pare la ejecucion
divide("2", "1")
```

Devolución de varios elementos

- En Python, al contrario que en otros lenguajes de programación, es posible devolver más de un elemento.
- En ese caso, la devolución se realiza con un **`return`** que tiene como parámetros varios valores separados por comas.
- En el siguiente ejemplo, se utiliza esta característica para obtener tres datos introducidos por teclado y asignárselos a tres variables nombre, apellidos y edad.

```
#Tema4_11
def pide_datos_persona():
    nombre2 = input("Dame tu nombre: ")
    apellidos2 = input("Dame tus apellidos: ")
    edad2 = int(input("Dame tu edad: "))
    return nombre2, apellidos2, edad2

nombre, apellidos, edad = pide_datos_persona()
print(nombre, apellidos, edad)
```

Funciones

Invocación de funciones con varios argumentos Asignación posicional

- Al invocar una función con varios argumentos, la asignación parámetro-argumento se realiza por posición. A cada parámetro se le asignará el valor del argumento correspondiente dado su orden en la invocación.

```
def area_rectangulo(altura, anchura):
    return altura*anchura
#parámetros formales separados por ,
area = area_rectangulo(3,4) #altura=3, anchura=4
```

Asignación clave=valor

- También se puede invocar mediante asignaciones con la forma *parámetro=argumento*. Este método permite crear correspondencias en un orden diferente al definido

```
area = area_rectangulo(anchura=3, altura=4)
```

Valores por defecto para los argumentos

- Es posible asignar valores por defecto a los parámetros de una función.
- Se usará el valor por defecto en caso de que al invocar la función no se asigne un argumento al parámetro correspondiente.
- Permite llamar a las funciones con menos valores de los definidos ganando en flexibilidad

```
def calculo_iva (precio, iva=21):
    impuesto=(precio*iva)/100
    return precio+impuesto

Ordenador=1000
print(calculo_iva(ordenador))
pan=1
print(calculo_iva(pan, 4))
#equivalente calculo_iva(precio=pan, iva=4)
```

Funciones anidadas (privadas)

- Otra capacidad de Python es la posibilidad de crear **funciones privadas**, es decir, funciones auxiliares para una principal, de tal forma que estas funciones auxiliares sean inaccesibles desde el exterior.

```
#Tema4_12
def pide_datos_persona():
    def input_int(msg):
        return int(input(msg))
    nombre = input("Dame tu nombre: ")
    apellidos = input("Dame tus apellidos: ")
    edad = input_int("Dame tu edad: ")
    return nombre, apellidos, edad

nombre, apellidos, edad = pide_datos_persona()
print(nombre, apellidos, edad)
a=input_int("hola")
```

Documentación

- Python soporta las llamadas **docstrings**, que son cadenas especificadas mediante as triples comillas, y contienen información sobre una entidad del código, en este caso, una función. En cualquier caso, siempre se sitúa antes de la primera sentencia. Esta documentación está disponible posteriormente mediante la función **help().**

```
#Tema4_13
def pide_datos_persona():
    """Devuelve el nombre, apellidos y edad."""
    nombre = input("Dame tu nombre: ")
    apellidos = input("Dame tus apellidos: ")
    edad = int(input("Dame tu edad: "))
    return nombre, apellidos, edad

help(pide_datos_persona)
```

```
Help on function pide_datos_persona in module __main__:
pide_datos_persona()
    Devuelve el nombre, apellidos y edad.
```

Funciones

- Las **docstring** son informaciones de las que se espera que se sigan varios criterios a la hora de crearlas:
  1. Deben empezar por mayúscula y terminar en un punto.
  2. Deben usarse siempre las triples comillas.
  3. Nunca se repite el nombre de la función, ni se repite una especie de firma de la misma.
  4. La documentación para los parámetros lleva como prefijo: ":**param nombre>**:", donde nombre es el nombre del parámetro.
  5. Los valores de retorno se prefijan como ":**return**:"
- En el caso de necesitar más de una línea, se deja una línea en blanco entre lo que pasa a ser el resumen, y la explicación más elaborada.
- Las triples comillas de cierre siempre se colocan en su propia línea.

```
#Tema4_14
def pide_datos_persona():
    """Devuelve el nombre, apellidos y edad, tras pedirlos por
 teclado.

    :return: lo siguiente, en el mismo orden
        nombre -- el nombre de la persona, como cadena
        apellidos -- los apellidos de la persona, como cadena
        edad -- la edad de la persona, como entero.
    """

    nombre = input("Dame tu nombre: ")
    apellidos = input("Dame tus apellidos: ")
    edad = int(input("Dame tu edad: "))
    return nombre, apellidos, edad


help(pide_datos_persona)
```

Funciones

Variables locales y globales

- En el cuerpo de las funciones es posible definir y usar variables. Vamos a estudiar propiedades de las variables definidas en el cuerpo de una función y en qué se diferencian de las variables que definimos fuera de cualquier función, es decir, en el denominado **programa principal**.

```
#Tema4_15
from math import sqrt
def area_triangulo(a,b,c):
    s = (a+b+c)/2.0
    return sqrt(s*(s-a)*(s-b)*(s-c))

area = area_triangulo(1,3,2.5)
print(area)
print(s)
 1.1709371246996996
 Traceback (most recent call last):
 File "area_triangulo.py", line 9, in <module>
 print(s)
 NameError: name 's' is not defined
```

- Este programa calcula el área de un triángulo conocida la longitud de sus lados a, b, c
- Las variables que sólo existen en el cuerpo de una función se denominan **variables locales**. En contraposición, el resto de variables se llaman **variables globales**.
- También los parámetros formales de una función se consideran **variables locales**, así que no puedes acceder a su valor fuera del cuerpo de la función. ¿y cuándo se crean a, b y c?
  - Cuando llamamos a la función con, por ejemplo, **`area_triangulo(1,3,2.5)`**
  - los parámetros a, b y c se crean como variables locales en la función y con los valores 1, 3 y 2.5, respectivamente.
  - Al acabar la ejecución de la función, las variables locales a, b y c dejan de existir (del mismo modo que deja de existir la variable local s).

Funciones

El mecanismo de las llamadas a función

- Cada vez que se produce una llamada, la ejecución del programa principal o de la función «actual» queda suspendida a la espera de que finalice la llamada realizada y prosigue cuando esta finaliza.
  - ¿Cómo recuerda Python qué funciones están «suspendidas» y en qué orden deben reanudarse?
- Por otra parte, hemos visto que si una variable local a una función tiene el mismo nombre que una variable global, durante la ejecución de la función la variable local oculta a la global y su valor es inaccesible.
  - ¿Cómo es posible que al finalizar la ejecución de una función se restaure el valor original? ¿Dónde se había almacenado este mientras la variable era invisible?

(Ver Introducción a la programación con Python 3, pag 263-267)

- Python utiliza internamente una estructura especial de memoria para recordar la información asociada a cada invocación de función: la **pila de llamadas a función**.

La pila de llamadas a función y el paso de parámetros

- Una **pila** es una serie de elementos a la que solo podemos añadir y eliminar componentes por la **cima**.
- Cada activación de una función apila un nuevo componente en la pila.
- Dicho componente, que recibe el nombre de **trama de activación**, es una zona de memoria en la que Python dispondrá espacio para los **punteros** asociados a parámetros, variables locales y otra información que se ha de recordar, como el punto exacto desde el que se efectuó la llamada a la función. Cuando iniciamos la ejecución de un programa, Python reserva una trama especial para las variables globales, así que empezamos con un elemento en la pila.

Funciones

Paso del resultado de expresiones como argumentos

- Hemos visto que el paso de parámetros comporta que el parámetro **apunte** a cierto lugar de la memoria. Pero ¿qué ocurre si pasamos una expresión como argumento?
- El parámetro formal apuntará entonces a una zona de memoria que almacena el resultado de la evaluación de la expresión

Más sobre el paso de parámetros

- ¿Qué ocurre si el parámetro es modificado dentro de la función? ¿Se modificará igualmente la variable o parámetro del ámbito desde el que se produce la llamada?

Veamos un **ejemplo**:

```
#Tema4_18
def incrementa(p):
    p=p+1
    return p

a=1
b=incrementa(a)
print('a: ',a)
print('b: ',b)
```

Salida:

```
a:  1
b:  2
```

Acceso a variables globales desde funciones

- Dentro de una función también se puede consultar y modificar variables globales. Tendremos que «avisar» a Python de que una variable usada en la función es **global**.
- Python utiliza una regla simple para decidir si una variable usada en una función es local o global: ***si se le asigna un valor, es local; si no, es global.***

Funciones

- Hay una forma de declarar en Python que unas variables son globales

```
#Tema4_19
a=1
def incrementa(p):
    global a #avisa de que a es global
    a=a+1   #modifica variable global a
    p=p+1
    return p
print(incrementa(2))
print(a)
```

```
3
2
```

- **Modificar variables globales** desde una función **no es** una práctica de programación recomendable. Sólo en contadas ocasiones está justificado que una función modifique variables globales.

```
def subrutina():
    global a
    print(a)
    a = 1
    return

a = 5
subrutina()
print(a)
```

```
5
1
```

```
#Tema4_20
mi_variable_global = 20

def una_funcion():
    global mi_variable_global #En este caso sí vamos a
 modificar a la global
    mi_variable_global=10 #Ahora la variable es 10, no
 20
    print("Una función, la variable global es: " + str
(mi_variable_global))

def otra_funcion():
    print("Otra función, la variable global es: " + st
r(mi_variable_global))

una_funcion()
otra_funcion()
```

```
Una función, la variable global es: 10
Otra función, la variable global es: 10
```

Crear nuestros propios Módulos

- De la misma forma que las funciones son grupos de instrucciones que se ejecutan con frecuencia, los módulos son grupos de funciones que se ejecutan, de forma relacionada, también con frecuencia.
- A lo largo de estos apuntes, se han utilizado a menudo módulos de la librería estándar, como **`random`** o **`math`**.
- Crear nuestros propios módulos no es nada complicado: de hecho, un archivo con extensión .py ya es un módulo en sí mismo.
- Un módulo sencillo es simplemente un archivo .py (**miModulo.py**) donde el nombre del archivo indica el nombre del módulo.
- Para usar **miModulo.py** dentro de un programa, el archivo debe estar en el mismo directorio.
  - El uso de un módulo definido por el usuario es igual al uso de un módulo de Python, y por tanto sigue la misma sintaxis que los módulos de Python.
  - El único problema es que, al importarlo, se ejecutará el código "libre" (es decir, que no pertenezca a una función) que haya en su interior.
  - La solución está en la variable **`__name__`**, como se ve a continuación.

Funciones

```
#pitagoras.py
#Módulo de funciones trigonométricas
#Basadas en el teorema de Pitágoras
import math

def calcula_hipotenusa(c1,c2):
    return math.sqrt((c1**2)+(c2**2))

def calcula_distancia_euclidea(x1, y1, x2, y2):
    distancia_x=math.sqrt((x1-x2)**2)
    distancia_y=math.sqrt((y1-y2)**2)

    return calcula_hipotenusa(distancia_x, distancia_y))

if __name__=="__main__":
    print("Pitagoras para 3, 4:", calcula_hipotenusa(3,4))
    print("Distancia de 1, 1 a origen:",calcula_distancia_
euclidea(0,0,3,4))
else:
    print(f"El módulo {__name__} ha sido importado.")
```

- Este módulo se podría utilizar en el programa siguiente:

```
#Tema4_21
# Calcula la hipotenusa
import pitagoras

C1=float(input("Dame el tamaño del primer Cateto: "))
C2=float(input("Dame el tamaño del segundo Cateto: "))

print("Para los Catetos ", C1,' y ', C2," la hipotenusa es:
", pitagoras.calcula_hipotenusa(C1, C2))
```

- La variable **__name__** toma el nombre dado cuando se importó el programa como un módulo. Es decir, cuando se importa **pitágoras** en el programa más arriba, la variable **__name__** dentro de pitagoras.py toma el valor "**pitagoras**".
- Sin embargo, si se ejecuta directamente **pitagoras.py**, entonces la variable **__name__**toma el valor "**__main__**".

- De ahí que comparando con este último valor sea posible saber si el módulo se está importando o ejecutando directamente.

```
# mimodulo
def f1():
    print("f1")
def f2():
    print("f2")

if __name__=="__main__":
    print("Ejecutando mimodulo:")
    f1()
    f2()
```

```
Ejecutando mimodulo:
f1
f2
```

Integración numérica

- Vamos a implementar un programa de integración numérica que aproxime el valor de

$$\int_a^b x^2\,dx$$

- Con la fórmula

$$\sum_{i=a}^{b} \Delta x\,(a + i\Delta x)^2$$

donde $\Delta x = (b - a)/n$

- El valor de **n** lo proporcionamos nosotros: a mayor valor de **n**, mayor precisión en la aproximación. Este método de aproximación de integrales se basa en el cálculo del área de una serie de rectángulos.

- La función que vamos a definir será **`integral_x2`** y necesita tres parámetros: el extremo izquierdo del intervalo (**a**), el derecho (**b**) y el número de rectángulos con los que se efectúa la aproximación (**n**).

Ejemplos con Funciones

```
#Tema4_22
def integral_x2 (a,b,n):
    if n==0:
        sumatorio=0.0
    else:
        deltax=(b-a)/n
        sumatorio=0.0
        for i in range(n):
            sumatorio+=deltax*(a+i*deltax)**2
    return sumatorio

inicio=float(input('Inicio del intervalo: '))
final=float(input('Final del intervalo: '))
partes=int(input('Número de rectángulos: '))
print('La integral de x**2 entre %3.2f y %3.2f ' %(inicio,final), end=' ')
print('vale aproximadamente {0}'.format(integral_x2(inicio,final,partes)))
```

$$\int_a^b x^2\,dx$$

$$\sum_{i=a}^{b} \Delta x\,(a+i\Delta x)^2$$

donde $\Delta x = (b-a)/n$

Ejemplos con Funciones

Integración numérica

- El método de integración que hemos implementado presenta un inconveniente: solo puede usarse para calcular la integral definida de una sola función: **$f(x)=x^2$**.
- Pero si queremos integrar por ejemplo **$g(x)=x^3$**, ¿tenemos que volver a escribir todo el programa?
- ***¡Python nos permite pasar funciones como argumentos!.*** Cuando pasamos la función como parámetro, no usamos paréntesis con argumentos; solo pasamos el nombre de la función.
- El nombre de la función es una **variable que contiene una referencia** a una zona de memoria en la que se encuentran las instrucciones que hemos de ejecutar al llamar a la función.

```
#Tema4_23
def cuadrado(x):
    return x**2
def cubo (x):
    return x**3
def integral_definida (f,a,b,n):
    if n==0:
        sumatorio=0.0
    else:
        deltax=(b-a)/n
        sumatorio=0.0
        for i in range(n):
            sumatorio+=deltax*f(a+i*deltax)
    return sumatorio

inicio=float(input('Inicio del intervalo: '))
final=float(input('Final del intervalo: '))
partes=int(input('Número de rectangulos: '))

print('La integral de x**2 entre %3.2f y %3.2f es %7.3f '
 %(inicio,final,integral_definida(cuadrado,inicio, final,
partes)))

print('La integral de x**3 entre {0} y {1} es {2}: '.form
at(inicio,final,integral_definida(cubo,inicio, final,part
es)))
```

Ejemplos con Funciones

Aproximaciones serie de Taylor

- Hacer un programa con una función Taylor para calcular el **seno(x)**, otra para el **cos(x)**, otra para el **ln(1+x)** y por último para e^x según quiera el usuario, mediante series de Taylor

$$seno(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1}$$

$$cos(x) = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n)!} x^{2n}$$

$$ln(1+x) = \sum_{n=1}^{\infty} \frac{(-1)^{n+1}}{n} x^n \ \ si\ |x| < 1$$

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$$

(Se deja para el alumno la realización del ln(1+x) y de e^x)

```
#Tema4_24
from math import factorial,pow,pi,sin,cos
def senox(x,epsilon):
    termino=x
    n=1
    seno=0.0
    signo=1
    while termino>epsilon:
        seno+=termino*signo
        termino=pow(x,2*n+1)/factorial(2*n+1)
        n+=1
        signo*=-1
    return seno

def cosenox(x,epsilon):
    termino=1.0
    n=1
    coseno=0.0
    signo=1
    while termino>epsilon:
        coseno+=termino*signo
        termino=pow(x,2*n)/factorial(2*n)
        n+=1
        signo*=-1
    return coseno

angulo=float(input('Introduzca el ángulo: '))
angulo*=pi/180
error=float(input('Introduzca el error: '))
print('El seno por Taylor es ',senox(angulo,error),' y con sin es
',sin(angulo))
print('\n\n El coseno por Taylor es ',cosenox(angulo,error),' y co
n cos es ',cos(angulo))
```

Ejemplos con Funciones

Cálculo de números combinatorios

- Ahora vamos a diseñar una función que calcule de cuántas formas podemos escoger k elementos de un conjunto con n objetos. Recuerda que la fórmula es:

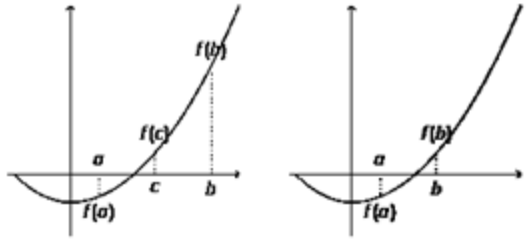
$$\binom{n}{k} = \frac{n!}{(n-k)!k!}$$

```
#Tema4_25
def fact(x):
    if x < 2:
        return 1
    else:
        f = 1
        for i in range(x,1,-1):
            f = f*i
        return f
def combinaciones(n,k):
    return fact(n)/(fact(n-k)*fact(k))

n = int(input("Introduce n: "))
k = int(input("Introduce k: "))
print('Las combinaciones de %d elementos tomados de %d en %d so
n %5.2f' %(n,k,k,combinaciones (n,k)))
```

El método de la bisección

- El método de la bisección permite encontrar un cero de una función matemática f(x) en un intervalo [a,b] si f(x) es continua en dicho intervalo y f(a) y f(b) son de distinto signo




```
#Tema4_26 f(x)=x**2-2*x-2 en [0.5,3.5]
def f(x):
    return x**2-2*x-2
def biseccion(a,b,epsilon):
    c=(a+b)/2
    while f(c)!=0 and b-a > epsilon:
        if f(a)*f(c)>0:
            a=c
        elif f(b)*f(c)>0:
            b=c
        c=(a+b)/2
    return c

print('El cero esta en: ',biseccion(0.5,3.5,1e-5))
```

Diseño de programas con funciones

Recursión

- Una función que se llama a sí misma, directa o indirectamente, es una **función recursiva**. La recursión es un potente concepto con el que se pueden expresar ciertos procedimientos de cálculo muy elegantemente.
- No obstante, al principio cuesta un poco entender las funciones recursivas… y un poco más diseñar nuestras propias funciones recursivas. La recursión es un concepto difícil cuando estás aprendiendo a programar.

Cálculo recursivo del factorial

- Partiremos de la siguiente definición matemática, válida para valores positivos de n

$$n! = \begin{cases} 1, & \text{si } n = 0 \text{ o } n = 1; \\ n \cdot (n-1)!, & \text{si } n > 1. \end{cases}$$

Ver explicación Introducción a la programación con Python, pag 305-307

Funciones

```python
#Tema4_28
def fact(n):
    if n==0 or n==1:
        return 1
    elif n>1:
        return n*fact(n-1)

a=int(input('Dame un entero positivo: '))
print(fact(a))
```

Los números de Fibonacci

- Los dos primeros números de la secuencia valen 1 y cada número a partir del tercero se obtiene sumando los dos anteriores. Podemos expresar esta definición matemáticamente así:

$$F_n = \begin{cases} 1, & \text{si } n = 1 \text{ o } n = 2; \\ F_{n-1} + F_{n-2}, & \text{si } n > 2. \end{cases}$$

Ver explicación Introducción a la programación con Python, pag 310-311

```python
#Tema4_29
def fibonacci(n):
    if n==1 or n==2:
        return 1
    elif n>2:
        return fibonacci(n-1)+fibonacci(n-2)

a=int(input('Dame un entero positivo: '))
print(fibonacci(a))
```

- Los números de Fibonacci son bastante curiosos, pues aparecen espontáneamente en la naturaleza: abejas, conejos, plantas, espirales del girasol, la piel de las piñas, …



El algoritmo de Euclides

- Vamos a calcular el máximo común divisor (**mcd**) de dos números **m** y **n** por el Algoritmo de Euclides:
  - «Calcula el resto de dividir el mayor de los dos números por el menor de ellos. Si el resto es cero, entonces el máximo común divisor es el menor de ambos números.
  - Si el resto es distinto de cero, el máximo común divisor de **n** y **m** es el máximo común divisor de otro par de números: el formado por el menor de **n** y **m** y por dicho resto».

```
#Tema4_30
def min(a,b):
    if a<b:
        return a
    else:
        return b
def max(a,b):
    if a>b:
        return a
    else:
        return b

def mcd(m,n):
    menor=min(m,n)
    mayor=max(m,n)
    resto=mayor % menor
    if resto==0:
        return menor
    else:
        return mcd(menor,resto)

a=int(input('Dame un entero positivo: '))
b=int(input('Dame un entero positivo: '))
print('El Máximo común divisor de {0} y {1} es {2}'.for
mat(a,b,mcd(a,b)))
```

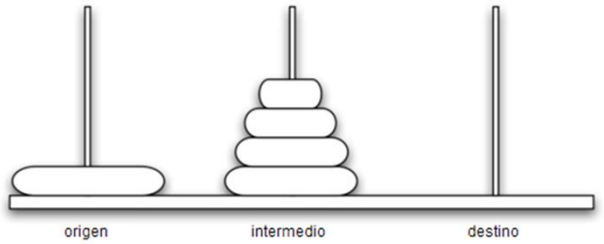
#Se podría obviar la definición de estas dos funciones y utilizar min() y max() predefinidas de Python

Funciones

Torres de Hanoi

Resolver el problema de las torres de Hanoi con discos que hemos de transferir de la aguja inicial a la aguja final requiere:

Las reglas básicas son las siguientes:
1.Los discos se apilan en orden de tamaño decreciente, con el disco más grande en la parte inferior y el más pequeño en la parte superior.
2.Solo se puede mover un disco a la vez.
3.Nunca se puede colocar un disco más grande sobre uno más pequeño.



- Resolver el problema con **n** discos requiere:
  - resolver el problema de las torres de Hanoi con **n−1** discos de la aguja inicial a la aguja libre
  - mover el último disco de la aguja inicial a la aguja de destino
  - y resolver el problema de las torres de Hanoi con **n−1** discos de la aguja libre a la aguja final.

```
#Tema4_31
def resuelve_Hanoi(n,inicial,final,libre):
    if n==1:
        print('Mover disco superior de aguja ',inicial,'
a ',final)
    else:
        resuelve_Hanoi(n-1,inicial,libre, final)
        print('Mover disco superior de aguja ',inicial,'
a ',final)
        resuelve_Hanoi(n-1,libre, final,inicial)

resuelve_Hanoi(3,1,3,2)
```

```
Mover disco superior de aguja  1  a  3
Mover disco superior de aguja  1  a  2
Mover disco superior de aguja  3  a  2
Mover disco superior de aguja  1  a  3
Mover disco superior de aguja  2  a  1
Mover disco superior de aguja  2  a  3
Mover disco superior de aguja  1  a  3
```